

Результаты

P	T(P), секунд
1	0.55
2	0.27
4	0.13
8	0.13
12	0.15
16	0.14

Резкое сокращение времени при увеличении числа процессов с 1 до 4 происходит за счёт того, что работа (умножение локального блока матрицы на соответствующий фрагмент вектора) делится между несколькими процессами.

Неравномерный рост производительности при дальнейших увеличениях числа процессов (8, 12, 16) связан с «oversubscribe» — это означает, что процессов больше, чем физических (или даже логических) ядер в системе, и они начинают конкурировать за ресурсы, контекст переключается чаще.



Пример работы программы:

Вывод:

Process 1/16: row_coord=0 col_coord=1, Ny=2500 Nx=2500

Process 1: A[0..4] = 0.346 0.598 0.317 0.920 0.833

Process 14/16: row_coord=3 col_coord=2, Ny=2500 Nx=2500

Process 14: A[0..4] = 0.003 0.965 0.748 0.078 0.471

Process 2/16: row_coord=0 col_coord=2, Ny=2500 Nx=2500

Process 2: A[0..4] = 0.703 0.508 0.117 0.237 0.263

Process 8/16: row_coord=2 col_coord=0, Ny=2500 Nx=2500

Process 8: A[0..4] = 0.851 0.986 0.432 0.655 0.866

Process 13/16: row_coord=3 col_coord=1, Ny=2500 Nx=2500

Process 13: A[0..4] = 0.145 0.052 0.446 0.759 0.538

Process 9/16: row_coord=2 col_coord=1, Ny=2500 Nx=2500

Process 9: A[0..4] = 0.209 0.898 0.236 0.976 0.799

Process 4/16: row_coord=1 col_coord=0, Ny=2500 Nx=2500

Process 4: A[0..4] = 0.918 0.333 0.221 0.376 0.630

Process 0/16: row_coord=0 col_coord=0, Ny=2500 Nx=2500

Process 0: A[0..4] = 0.487 0.684 0.015 0.098 0.897

Process 12/16: row_coord=3 col_coord=0, Ny=2500 Nx=2500

Process 12: A[0..4] = 0.279 0.132 0.636 0.429 0.095

Process 15/16: row_coord=3 col_coord=3, Ny=2500 Nx=2500

Process 15: A[0..4] = 0.861 0.379 0.052 0.898 0.407

Process 7/16: row_coord=1 col_coord=3, Ny=2500 Nx=2500

Process 7: A[0..4] = 0.992 0.073 0.126 0.335 0.933

Process 3/16: row_coord=0 col_coord=3, Ny=2500 Nx=2500

Process 3: A[0..4] = 0.060 0.924 0.921 0.057 0.201

Process 5/16: row_coord=1 col_coord=1, Ny=2500 Nx=2500

Process 5: A[0..4] = 0.782 0.753 0.029 0.702 0.072

Process 11/16: row_coord=2 col_coord=3, Ny=2500 Nx=2500

Process 11: A[0..4] = 0.924 0.222 0.337 0.113 0.665
Process 10/16: row_coord=2 col_coord=2, Ny=2500 Nx=2500
Process 10: A[0..4] = 0.069 0.812 0.538 0.297 0.235
Process 6/16: row_coord=1 col_coord=2, Ny=2500 Nx=2500
Process 6: A[0..4] = 0.634 0.659 0.326 0.016 0.498
Global b (first 5 elements): 0.068 0.723 0.971 0.360 0.021
Process 9: b_local[0..4] = 0.908 0.357 0.476 0.490 0.496
Process 3: b_local[0..4] = 0.516 0.670 0.001 0.209 0.205
Process 10: b_local[0..4] = 0.470 0.275 0.347 0.146 0.822
Process 15: b_local[0..4] = 0.516 0.670 0.001 0.209 0.205
Process 2: b_local[0..4] = 0.470 0.275 0.347 0.146 0.822
Process 13: b_local[0..4] = 0.908 0.357 0.476 0.490 0.496
Process 11: b_local[0..4] = 0.516 0.670 0.001 0.209 0.205
Process 5: b_local[0..4] = 0.908 0.357 0.476 0.490 0.496
Process 6: b_local[0..4] = 0.470 0.275 0.347 0.146 0.822
Process 8: b_local[0..4] = 0.068 0.723 0.971 0.360 0.021
Process 1: b_local[0..4] = 0.908 0.357 0.476 0.490 0.496
Process 12: b_local[0..4] = 0.068 0.723 0.971 0.360 0.021
Process 0: b_local[0..4] = 0.068 0.723 0.971 0.360 0.021
Process 4: b_local[0..4] = 0.068 0.723 0.971 0.360 0.021
Process 14: b_local[0..4] = 0.470 0.275 0.347 0.146 0.822
Process 7: b_local[0..4] = 0.516 0.670 0.001 0.209 0.205
Process 14: c[0..4] = 634.515 630.543 630.369 658.433 631.759
Process 13: c[0..4] = 615.152 616.200 606.175 606.299 607.664
Process 7: c[0..4] = 615.686 620.251 630.140 619.396 610.791
Process 9: c[0..4] = 607.796 621.103 604.123 617.300 637.467
Process 2: c[0..4] = 641.559 630.168 637.369 638.267 641.499
Process 11: c[0..4] = 622.833 631.294 623.370 627.776 609.074
Process 8: c[0..4] = 621.918 617.394 632.249 615.772 620.406
Process 0: c[0..4] = 628.662 613.108 613.096 606.406 615.582
Process 4: c[0..4] = 622.609 628.340 615.983 624.953 619.215
Process 6: c[0..4] = 628.883 639.320 624.827 628.399 613.452

Process 15: c[0..4] = 633.252 611.008 606.622 624.951 613.699

Process 3: c[0..4] = 605.402 608.979 609.133 628.097 612.329

Process 10: c[0..4] = 635.939 647.419 639.718 643.590 645.354

Process 1: c[0..4] = 629.927 628.858 627.934 609.969 631.575

Process 12: c[0..4] = 632.782 636.324 623.825 618.220 626.408

Process 5: c[0..4] = 604.380 628.905 624.894 598.565 629.533

==== Final results on rank 0 ====

c_global[0..4] = 2505.549 2481.113 2487.532 2482.739 2500.985

Time (total) = 0.199436

Time (calc only) = 0.118825

Пояснение:

Разбиение матрицы A и начальный вывод на каждом процессе

Process 0/16: row_coord=0 col_coord=0, Ny=2500 Nx=2500

Process 0: A[0..4] = 0.487 0.684 0.015 0.098 0.897

...

показывают:

1. **Номер процесса (rank) и всего процессов (size = 16).**
2. **Координаты** процесса в двумерной решётке:
 - row_coord — какой по счёту блок по вертикали (от 0 до 3, так как всего 4 строки блоков).
 - col_coord — какой по счёту блок по горизонтали (от 0 до 3, так как всего 4 столбца блоков).
3. Размеры локального блока матрицы:
 - Ny=2500 строк,
 - Nx=2500 столбцов.
4. Печать нескольких **первых элементов** массива A (всего печатается 5 элементов, A[0 . . 4]). Это случайные числа, сгенерированные на каждом процессе.

Матрица A общего размера 20000×20000 разбита на 16 блоков по схеме 4×4, так что у каждого процесса получается блок размером 2500×2500.

Вектор b

Далее, на процессе 0 формируется глобальный вектор b длиной N=20000.

Вывод:

Global b (first 5 elements): 0.068 0.723 0.971 0.360 0.021

Означает, что первые 5 элементов b — 0.068,0.723,0.971,0.360,0.021 (всего там 20,000 элементов, печатается только часть).

Затем каждый процесс считывает свой «локальный» отрезок b размером $N_x=2500$ (чтобы умножать на свои 2500 столбцов). Вывод строк вида:

```
Process 9: b_local[0..4] = 0.908 0.357 0.476 0.490 0.496
...
```

показывает, какие именно **первые 5 элементов** вектора b были вычитаны (через `MPI_Get`) на данном процессе.

Можно заметить, что процессы с `col_coord=0` получают элементы $b[0 \dots N_x-1]$ (т. е. сдвиг 0), процессы с `col_coord=1` получают диапазон $b[N_x \dots 2N_x-1]$, и т. д. Некоторые процессы при печати «`b_local`» имеют одни и те же первые 5 значений, поскольку одинаковый сдвиг (это бывает, если данные в логе упорядочены не по `rank`'ам, а по времени вывода, либо по особенностям `rand`).

Локальный результат c

Затем каждый процесс выполняет умножение своего локального блока матрицы $A(2500 \times 2500)$ на локальную часть $b(2500 \times 1)$ и получает вектор $c(2500 \times 1)$.

Строки вида:

```
Process 14: c[0..4] = 634.515 630.543 630.369 658.433 631.759
...
Process 0: c[0..4] = 628.662 613.108 613.096 606.406 615.582
...
```

показывают **первые 5 элементов** локального вектора c на каждом процессе. Значения в районе 600–650 говорят о том, что суммы произведений накопились в достаточно большом диапазоне.